

Trabajo Practico Grupal Mapeo Objeto Relacional – ORM – JPA - Hibernate

INTEGRANTES:

- Acosta, Lautaro;
- Canale, Jerónimo;
- Navarrete, Leandro;
- Pedraza, Marcos;
- Pereyra, Facundo;
- Septfons, Estefanía.

CURSO: 3K09 (Año 2026)

El objetivo de esta práctica es implementar el mapeo objeto-relacional (ORM) mediante JPA/Hibernate sobre el modelo de dominio provisto y configurar el motor de persistencia hacia una base de datos relacional.

1. Anotaciones JPA e Integridad del Modelo

- Configurar EntityId y AuditoriaApp como superclases mapeadas (@MappedSuperclass).
- Definir la clave primaria id en EntityId usando @Id y la estrategia de generación de IDs (@GeneratedValue).
- Mapear cada clase concreta con @Entity y @Table(name = "...").
- Aplicar @Column(nullable = false) a todos los campos marcados como obligatorios (no null).
- Definir el tipo de mapeo temporal (@Temporal o tipos java.time) para las fechas.
- Configurar correctamente las asociaciones entre entidades:
 - @ManyToOne: En auditoría (Usuario), FacturaVenta (PuntoVenta), FacturaVentaDetalle (FacturaVenta, ListaPrecioArticulo), ListaPrecioArticulo (ListaPrecio, Articulo) y Articulo (Rubro, Marca).
 - @OneToMany: En FacturaVenta hacia FacturaVentaDetalle (configurando mappedBy y cascade = CascadeType.ALL).
 - @OneToOne: En Cliente hacia Contacto y Domicilio.

2. Configuración de la Unidad de Persistencia

- Crear el archivo persistence.xml en la ruta src/main/resources/META-INF/.
- Definir una unidad de persistencia nombrada (name="FacturacionPU").
- Configurar el proveedor org.hibernate.jpa.HibernatePersistenceProvider.
- Establecer la conexión hacia MySQL o PostgreSQL definiendo las propiedades estándar:
 - Driver JDBC (javax.persistence.jdbc.driver)
 - URL de conexión (javax.persistence.jdbc.url)
 - Credenciales de acceso (user y password)

- Propiedades de Hibernate: `hibernate.show_sql = true`, `hibernate.format_sql = true` y `hibernate.hbm2ddl.auto = update`.

3. Persistencia en Cascading desde el Método Main

- Crear la clase principal Main con la firma `public static void main(String[] args)`.
- Iniciar el contenedor de JPA mediante `Persistence.createEntityManagerFactory("FacturacionPU")` y obtener el `EntityManager`.
- Iniciar una transacción (`em.getTransaction().begin()`).
- Instanciar los objetos necesarios (ej. `Usuario`, `PuntoVenta`, `Articulo`, `ListaPrecioArticulo`, `FacturaVenta`, etc.).
- Crear una cabecera de `FacturaVenta` y asignarle uno o más ítems `FacturaVentaDetalle` asociándolos bidireccionalmente.
- Requisito clave: Persistir únicamente el objeto cabecera `FacturaVenta` utilizando un solo llamado a `em.persist(facturaVenta)`.
- Verificar que, gracias a la configuración de cascada (`CascadeType.ALL / PERSIST`), se inserten automáticamente tanto la factura como todos sus detalles vinculados en la base de datos.
- Finalizar la transacción con `em.getTransaction().commit()` y cerrar el `EntityManager` y `EntityManagerFactory`.

Modelo de Clases Entidad

```
// =====
// Superclases / Herencia
// =====

// TODO: Agregar anotación @MappedSuperclass
public abstract class EntityId {
// TODO: Agregar @Id y @GeneratedValue(strategy = GenerationType.IDENTITY)
protected Long id;
}

// TODO: Agregar anotación @MappedSuperclass
public abstract class AuditoriaApp extends EntityId {
// TODO: Configurar @Column(nullable = false) y formato de fecha
protected Date fechaAlta;
protected Date fechaBaja;

// TODO: Configurar @Column(nullable = false) y formato de fecha
protected Date fechaModificacion;

// TODO: Configurar @ManyToOne y @JoinColumn(nullable = false)
protected Usuario usuarioCarga;
```

```

// TODO: Configurar @ManyToOne
protected Usuario usuarioBaja;

// TODO: Configurar @ManyToOne y @JoinColumn(nullable = false)
protected Usuario usuarioModificacion;

}

// =====
// Entidades del Dominio
// =====

// TODO: Agregar @Entity y @Table
public class Usuario extends EntityId {
// TODO: Configurar @Column(nullable = false) en los 4 atributos
private String usuario;
private String clave;
private String nombre;
private String apellido;
}

// TODO: Agregar @Entity y @Table
public class FacturaVenta extends AuditoriaApp {
private Long numero;

// TODO: Configurar @Column(nullable = false)
private Date fechaEmision;

// TODO: Configurar @ManyToOne y @JoinColumn(nullable = false)
private PuntoVenta puntoVenta;

private double importeCobrado;
private double importeSaldo;

// TODO: Configurar @Column(nullable = false)
private double importeTotal;

private String cae;
private Date caeFechaVencimiento;
private String resultadoAfip;
private String motivoRechazo;

// TODO: Configurar @Column(nullable = false)
private String estado;

private Date fechaAnulacion;
private String observaciones;

```

```
// TODO: Configurar @OneToMany(mappedBy = "factura", cascade = CascadeType.ALL)
private List<FacturaVentaDetalle> detalles;
```

```
}
```

```
// TODO: Agregar @Entity y @Table
public class PuntoVenta extends AuditoriaApp {
// TODO: Configurar @Column(nullable = false)
private int numero;
```

```
private String descripcion;
private String tipoEmision;
private String domicilioComercial;
```

```
}
```

```
// TODO: Agregar @Entity y @Table
public class TipoMoneda extends AuditoriaApp {
// TODO: Configurar @Column(nullable = false) en los 3 atributos
private String codigoAfip;
private String denominacion;
private String simbolo;
}
```

```
// TODO: Agregar @Entity y @Table
public class FacturaVentaDetalle extends EntityId {
// TODO: Configurar @ManyToOne y @JoinColumn(nullable = false)
private FacturaVenta factura;
```

```
// TODO: Configurar @ManyToOne y @JoinColumn(nullable = false)
private ListaPrecioArticulo listaPrecioArticulo;
```

```
private String descripcion;
```

```
// TODO: Configurar @Column(nullable = false) en cantidad, precioUnitario e importeSubtotal
private double cantidad;
private double precioUnitario;
private double porcentajeBonificacion;
private double importeNeto;
private double importeIva;
private double importeSubtotal;
```

```
}
```

```

// TODO: Agregar @Entity y @Table
public class Cliente extends AuditoriaApp {
// TODO: Configurar @Column(nullable = false) en cuitCuil y denominacion
private String cuitCuil;
private String denominacion;

// TODO: Configurar @OneToOne y @JoinColumn(nullable = false)
private Contacto contacto;

// TODO: Configurar @OneToOne y @JoinColumn(nullable = false)
private Domicilio domicilio;

}

// TODO: Agregar @Entity y @Table
public class CondicionIva extends AuditoriaApp {
// TODO: Configurar @Column(nullable = false) en codigoAfip y denominacion
private int codigoAfip;
private String denominacion;
}

// TODO: Agregar @Entity y @Table
public class Contacto extends EntityId {
private String email;
private String telefono;
private String celular;
}

// TODO: Agregar @Entity y @Table
public class Domicilio extends EntityId {
private String nombreCalle;
private String numeroCalle;
}

// TODO: Agregar @Entity y @Table
public class ListaPrecioArticulo extends AuditoriaApp {
// TODO: Configurar @ManyToOne y @JoinColumn(nullable = false)
private ListaPrecio listaPrecio;

// TODO: Configurar @Column(nullable = false)
private double precioVenta;

// TODO: Configurar @ManyToOne y @JoinColumn(nullable = false)
private Articulo articulo;

}

```

```

// TODO: Agregar @Entity y @Table
public class Artículo extends AuditoriaApp {
// TODO: Configurar @ManyToOne
private Rubro rubro;

// TODO: Configurar @Column(nullable = false) en codigo y denominacion
private String codigo;
private String denominacion;

// TODO: Configurar @ManyToOne
private Marca marca;

}

// TODO: Agregar @Entity y @Table
public class ListaPrecio extends AuditoriaApp {
// TODO: Configurar @Column(nullable = false) en codigo y denominacion
private String codigo;
private String denominacion;
}

// TODO: Agregar @Entity y @Table
public class Marca extends AuditoriaApp {
// TODO: Configurar @Column(nullable = false) en denominacion y codigo
private String denominacion;
private Integer codigo;
}

// TODO: Agregar @Entity y @Table
public class Rubro extends AuditoriaApp {
// TODO: Configurar @Column(nullable = false) en denominacion y codigo
private String denominacion;
private Integer codigo;
}

```

1)

Auditoria app

```

import jakarta.persistence.*;
import java.util.Date;

@MappedSuperclass
public abstract class AuditoriaApp extends EntityId {

    @Column(nullable = false)

```

```
@Temporal(TemporalType.TIMESTAMP)
protected Date fechaAlta;

@Temporal(TemporalType.TIMESTAMP)
protected Date fechaBaja;

@Column(nullable = false)
@Temporal(TemporalType.TIMESTAMP)
protected Date fechaModificacion;

@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(nullable = false)
protected Usuario usuarioCarga;

@ManyToOne(fetch = FetchType.LAZY)
protected Usuario usuarioBaja;

@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(nullable = false)
protected Usuario usuarioModificacion;

    public Date getFechaAlta() {
        return fechaAlta;
    }

    public void setFechaAlta(Date fechaAlta) {
        this.fechaAlta = fechaAlta;
    }

    public Date getFechaBaja() {
        return fechaBaja;
    }

    public void setFechaBaja(Date fechaBaja) {
        this.fechaBaja = fechaBaja;
    }

    public Date getFechaModificacion() {
        return fechaModificacion;
    }

    public void setFechaModificacion(Date fechaModificacion) {
        this.fechaModificacion = fechaModificacion;
    }
```

```

    }

    public Usuario getUsuarioCarga() {
        return usuarioCarga;
    }

    public void setUsuarioCarga(Usuario usuarioCarga) {
        this.usuarioCarga = usuarioCarga;
    }

    public Usuario getUsuarioBaja() {
        return usuarioBaja;
    }

    public void setUsuarioBaja(Usuario usuarioBaja) {
        this.usuarioBaja = usuarioBaja;
    }

    public Usuario getUsuarioModificacion() {
        return usuarioModificacion;
    }

    public void setUsuarioModificacion(Usuario usuarioModificacion) {
        this.usuarioModificacion = usuarioModificacion;
    }
}

```

EntityId

```

import jakarta.persistence.*;
import java.util.Date;

@MappedSuperclass
public abstract class EntityId {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    protected Long id;

    public Long getId() {
        return id;
    }
}

```

```

    }

    public void setId(Long id) {
        this.id = id;
    }
}

```

Articulo

```

import jakarta.persistence.*;

@Entity
@Table(name = "articulos")
public class Articulo extends AuditoriaApp {

    @ManyToOne(fetch = FetchType.LAZY)
    private Rubro rubro;

    @Column(nullable = false)
    private String codigo;

    @Column(nullable = false)
    private String denominacion;

    @ManyToOne(fetch = FetchType.LAZY)
    private Marca marca;

    public Rubro getRubro() {
        return rubro;
    }

    public void setRubro(Rubro rubro) {
        this.rubro = rubro;
    }

    public String getCodigo() {
        return codigo;
    }

    public void setCodigo(String codigo) {
        this.codigo = codigo;
    }

    public String getDenominacion() {
        return denominacion;
    }

    public void setDenominacion(String denominacion) {

```

```

        this.denominacion = denominacion;
    }
    public Marca getMarca() {
        return marca;
    }
    public void setMarca(Marca marca) {
        this.marca = marca;
    }
}

```

ListaPrecios

```

import jakarta.persistence.*;

@Entity
@Table(name = "listas_precios")
public class ListaPrecio extends AuditoriaApp {

    @Column(nullable = false)
    private String codigo;

    @Column(nullable = false)
    private String denominacion;

    public String getCodigo() {
        return codigo;
    }
    public void setCodigo(String codigo) {
        this.codigo = codigo;
    }
    public String getDenominacion() {
        return denominacion;
    }
    public void setDenominacion(String denominacion) {
        this.denominacion = denominacion;
    }
}

```

ListaPreciosDetalle

```

import jakarta.persistence.*;

@Entity
@Table(name = "listas_precios_articulos")

```

```

public class ListaPrecioArticulo extends AuditoriaApp {

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(nullable = false)
    private ListaPrecio listaPrecio;

    @Column(nullable = false)
    private double precioVenta;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(nullable = false)
    private Articulo articulo;

    public ListaPrecio getListaPrecio() {
        return listaPrecio;
    }
    public void setListaPrecio(ListaPrecio listaPrecio) {
        this.listaPrecio = listaPrecio;
    }
    public double getPrecioVenta() {
        return precioVenta;
    }
    public void setPrecioVenta(double precioVenta) {
        this.precioVenta = precioVenta;
    }
    public Articulo getArticulo() {
        return articulo;
    }
    public void setArticulo(Articulo articulo) {
        this.articulo = articulo;
    }
}

```

Marca

```

import jakarta.persistence.*;

@Entity
@Table(name = "marcas")
public class Marca extends AuditoriaApp {

    @Column(nullable = false)
    private String denominacion;

```

```

@Column(nullable = false)
private Integer codigo;

    public String getDenominacion() {
        return denominacion;
    }
    public void setDenominacion(String denominacion) {
        this.denominacion = denominacion;
    }
    public Integer getCodigo() {
        return codigo;
    }
    public void setCodigo(Integer codigo) {
        this.codigo = codigo;
    }
}

```

Rubro

```

import jakarta.persistence.*;

@Entity
@Table(name = "rubros")
public class Rubro extends AuditoriaApp {

    @Column(nullable = false)
    private String denominacion;

    @Column(nullable = false)
    private Integer codigo;

    public String getDenominacion() {
        return denominacion;
    }
    public void setDenominacion(String denominacion) {
        this.denominacion = denominacion;
    }
    public Integer getCodigo() {
        return codigo;
    }
    public void setCodigo(Integer codigo) {
        this.codigo = codigo;
    }
}

```

```
}  
}
```

FacturaVenta

```
import jakarta.persistence.*;  
import java.util.Date;  
import java.util.List;  
  
@Entity  
@Table(name = "facturas_ventas")  
public class FacturaVenta extends AuditoriaApp {  
  
    private Long numero;  
  
    @Column(nullable = false)  
    @Temporal(TemporalType.DATE)  
    private Date fechaEmision;  
  
    @ManyToOne(fetch = FetchType.LAZY)  
    @JoinColumn(nullable = false)  
    private PuntoVenta puntoVenta;  
  
    private double importeCobrado;  
    private double importeSaldo;  
  
    @Column(nullable = false)  
    private double importeTotal;  
  
    private String cae;  
  
    @Temporal(TemporalType.DATE)  
    private Date caeFechaVencimiento;  
  
    private String resultadoAfip;  
    private String motivoRechazo;  
  
    @Column(nullable = false)  
    private String estado;  
  
    @Temporal(TemporalType.TIMESTAMP)  
    private Date fechaAnulacion;
```

```
private String observaciones;

@OneToMany(mappedBy = "factura", cascade = CascadeType.ALL)
private List<FacturaVentaDetalle> detalles;

    public Long getNumero() {
        return numero;
    }
    public void setNumero(Long numero) {
        this.numero = numero;
    }
    public Date getFechaEmision() {
        return fechaEmision;
    }
    public void setFechaEmision(Date fechaEmision) {
        this.fechaEmision = fechaEmision;
    }
    public PuntoVenta getPuntoVenta() {
        return puntoVenta;
    }
    public void setPuntoVenta(PuntoVenta puntoVenta) {
        this.puntoVenta = puntoVenta;
    }
    public double getImporteCobrado() {
        return importeCobrado;
    }
    public void setImporteCobrado(double importeCobrado) {
        this.importeCobrado = importeCobrado;
    }
    public double getImporteSaldo() {
        return importeSaldo;
    }
    public void setImporteSaldo(double importeSaldo) {
        this.importeSaldo = importeSaldo;
    }
    public double getImporteTotal() {
        return importeTotal;
    }
    public void setImporteTotal(double importeTotal) {
        this.importeTotal = importeTotal;
    }
    public String getCae() {
        return cae;
    }
```

```
}  
public void setCae(String cae) {  
    this.cae = cae;  
}  
public Date getCaeFechaVencimiento() {  
    return caeFechaVencimiento;  
}  
public void setCaeFechaVencimiento(Date caeFechaVencimiento) {  
    this.caeFechaVencimiento = caeFechaVencimiento;  
}  
public String getResultadoAfip() {  
    return resultadoAfip;  
}  
public void setResultadoAfip(String resultadoAfip) {  
    this.resultadoAfip = resultadoAfip;  
}  
public String getMotivoRechazo() {  
    return motivoRechazo;  
}  
public void setMotivoRechazo(String motivoRechazo) {  
    this.motivoRechazo = motivoRechazo;  
}  
public String getEstado() {  
    return estado;  
}  
public void setEstado(String estado) {  
    this.estado = estado;  
}  
public Date getFechaAnulacion() {  
    return fechaAnulacion;  
}  
public void setFechaAnulacion(Date fechaAnulacion) {  
    this.fechaAnulacion = fechaAnulacion;  
}  
public String getObservaciones() {  
    return observaciones;  
}  
public void setObservaciones(String observaciones) {  
    this.observaciones = observaciones;  
}  
public List<FacturaVentaDetalle> getDetalles() {  
    return detalles;  
}  
}
```

```

    public void setDetalles(List<FacturaVentaDetalle> detalles) {
        this.detalles = detalles;
    }
}

```

FacturaVentaDetalle

```

import model.catalogo.ListaPrecioArticulo;
import jakarta.persistence.*;

@Entity
@Table(name = "facturas_ventas_detalles")
public class FacturaVentaDetalle extends EntityId {

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(nullable = false)
    private FacturaVenta factura;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(nullable = false)
    private ListaPrecioArticulo listaPrecioArticulo;

    private String descripcion;

    @Column(nullable = false)
    private double cantidad;

    @Column(nullable = false)
    private double precioUnitario;

    private double porcentajeBonificacion;
    private double importeNeto;
    private double importeIva;

    @Column(nullable = false)
    private double importeSubtotal;

    public FacturaVenta getFactura() {
        return factura;
    }

    public void setFactura(FacturaVenta factura) {
        this.factura = factura;
    }
}

```

```
public ListaPrecioArticulo getListaPrecioArticulo() {
    return listaPrecioArticulo;
}

public void setListaPrecioArticulo(ListaPrecioArticulo listaPrecioArticulo) {
    this.listaPrecioArticulo = listaPrecioArticulo;
}

public String getDescripcion() {
    return descripcion;
}

public void setDescripcion(String descripcion) {
    this.descripcion = descripcion;
}

public double getCantidad() {
    return cantidad;
}

public void setCantidad(double cantidad) {
    this.cantidad = cantidad;
}

public double getPrecioUnitario() {
    return precioUnitario;
}

public void setPrecioUnitario(double precioUnitario) {
    this.precioUnitario = precioUnitario;
}

public double getPorcentajeBonificacion() {
    return porcentajeBonificacion;
}

public void setPorcentajeBonificacion(double porcentajeBonificacion) {
    this.porcentajeBonificacion = porcentajeBonificacion;
}

public double getImporteNeto() {
    return importeNeto;
}

public void setImporteNeto(double importeNeto) {
    this.importeNeto = importeNeto;
}

public double getImporteIva() {
    return importeIva;
}

public void setImporteIva(double importeIva) {
    this.importeIva = importeIva;
}

public double getImporteSubtotal() {
```

```

        return importeSubtotal;
    }

    public void setImporteSubtotal(double importeSubtotal) {
        this.importeSubtotal = importeSubtotal;
    }
}

```

PuntoVenta

```

import jakarta.persistence.*;

@Entity
@Table(name = "puntos_ventas")
public class PuntoVenta extends AuditoriaApp {

    @Column(nullable = false)
    private int numero;

    private String descripcion;
    private String tipoEmision;
    private String domicilioComercial;

    public int getNumero() {
        return numero;
    }

    public void setNumero(int numero) {
        this.numero = numero;
    }

    public String getDescripcion() {
        return descripcion;
    }

    public void setDescripcion(String descripcion) {
        this.descripcion = descripcion;
    }

    public String getTipoEmision() {
        return tipoEmision;
    }

    public void setTipoEmision(String tipoEmision) {
        this.tipoEmision = tipoEmision;
    }

    public String getDomicilioComercial() {
        return domicilioComercial;
    }
}

```

```

    public void setDomicilioComercial(String domicilioComercial) {
        this.domicilioComercial = domicilioComercial;
    }
}

```

TipoMoneda

```

import jakarta.persistence.*;

@Entity
@Table(name = "tipos_monedas")
public class TipoMoneda extends AuditoriaApp {

    @Column(nullable = false)
    private String codigoAfip;

    @Column(nullable = false)
    private String denominacion;

    @Column(nullable = false)
    private String simbolo;

    public String getCodigoAfip() {
        return codigoAfip;
    }

    public void setCodigoAfip(String codigoAfip) {
        this.codigoAfip = codigoAfip;
    }

    public String getDenominacion() {
        return denominacion;
    }

    public void setDenominacion(String denominacion) {
        this.denominacion = denominacion;
    }

    public String getSimbolo() {
        return simbolo;
    }

    public void setSimbolo(String simbolo) {
        this.simbolo = simbolo;
    }
}

```

Cliente

```

import jakarta.persistence.*;

```

```
@Entity
@Table(name = "clientes")
public class Cliente extends AuditoriaApp {

    @Column(nullable = false)
    private String cuitCuil;

    @Column(nullable = false)
    private String denominacion;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(nullable = false)
    private Contacto contacto;

    @OneToOne(cascade = CascadeType.ALL)
    @JoinColumn(nullable = false)
    private Domicilio domicilio;

    public String getCuitCuil() {
        return cuitCuil;
    }
    public void setCuitCuil(String cuitCuil) {
        this.cuitCuil = cuitCuil;
    }
    public String getDenominacion() {
        return denominacion;
    }
    public void setDenominacion(String denominacion) {
        this.denominacion = denominacion;
    }
    public Contacto getContacto() {
        return contacto;
    }
    public void setContacto(Contacto contacto) {
        this.contacto = contacto;
    }
    public Domicilio getDomicilio() {
        return domicilio;
    }
    public void setDomicilio(Domicilio domicilio) {
        this.domicilio = domicilio;
    }
}
```

```
}
```

CondicionesIva

```
import jakarta.persistence.*;

@Entity
@Table(name = "condiciones_iva")
public class CondicionIva extends AuditoriaApp {

    @Column(nullable = false)
    private int codigoAfip;

    @Column(nullable = false)
    private String denominacion;

    public int getCodigoAfip() {
        return codigoAfip;
    }
    public void setCodigoAfip(int codigoAfip) {
        this.codigoAfip = codigoAfip;
    }
    public String getDenominacion() {
        return denominacion;
    }
    public void setDenominacion(String denominacion) {
        this.denominacion = denominacion;
    }
}
```

Contacto

```
import jakarta.persistence.*;

@Entity
@Table(name = "contactos")
public class Contacto extends EntityId {

    private String email;
    private String telefono;
    private String celular;

    public String getEmail() {
        return email;
    }
}
```

```

    }
    public void setEmail(String email) {
        this.email = email;
    }
    public String getTelefono() {
        return telefono;
    }
    public void setTelefono(String telefono) {
        this.telefono = telefono;
    }
    public String getCelular() {
        return celular;
    }
    public void setCelular(String celular) {
        this.celular = celular;
    }
}

```

Domicilio

```

import jakarta.persistence.*;

@Entity
@Table(name = "domicilios")
public class Domicilio extends EntityId {
    private String nombreCalle;
    private String numeroCalle;

    public String getNombreCalle() {
        return nombreCalle;
    }
    public void setNombreCalle(String nombreCalle) {
        this.nombreCalle = nombreCalle;
    }
    public String getNumeroCalle() {
        return numeroCalle;
    }
    public void setNumeroCalle(String numeroCalle) {
        this.numeroCalle = numeroCalle;
    }
}

```

Usuario

```
import jakarta.persistence.*;

@Entity
@Table(name = "usuarios")
public class Usuario extends EntityId {

    @Column(nullable = false)
    private String usuario;

    @Column(nullable = false)
    private String clave;

    @Column(nullable = false)
    private String nombre;

    @Column(nullable = false)
    private String apellido;

    public String getUsuario() {
        return usuario;
    }

    public void setUsuario(String usuario) {
        this.usuario = usuario;
    }

    public String getClave() {
        return clave;
    }

    public void setClave(String clave) {
        this.clave = clave;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getApellido() {
```

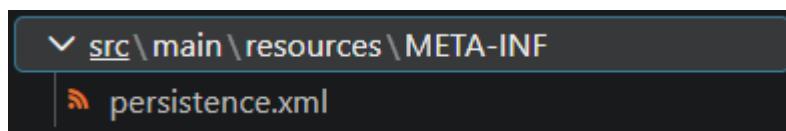
```

        return apellido;
    }

    public void setApellido(String apellido) {
        this.apellido = apellido;
    }
}

```

2)



```

<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
    version="3.0">

    <persistence-unit name="FacturacionPU"
transaction-type="RESOURCE_LOCAL">

        <!--proveedor-->

        <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>

        <!--propiedades-->

        <properties>

            <property name="javax.persistence.jdbc.driver"
value="com.mysql.cj.jdbc.Driver"/>

            <property name="javax.persistence.jdbc.url"
value="jdbc:mysql://localhost:3306/facturasDB"/>

```

```

<property name="javax.persistence.jdbc.user" value= "root"/>

<property name="javax.persistence.jdbc.password" value= ""/>

<!--Propiedades de Hibernate-->

<property name="hibernate.show_sql" value= "true"/>

<property name="hibernate.format_sql" value="true"/>

<property name= "hibernate.hbm2ddl.auto" value="update"/>

<!--cierre-->

</properties>

</persistence-unit>

</persistence>

```

3)

```

Main.java > Main > main(String[])
1  import jakarta.persistence.EntityManager;
2  import jakarta.persistence.EntityManagerFactory;
3  import jakarta.persistence.Persistence;
4  import java.util.ArrayList;
5  import java.util.Date;
6
7  public class Main {
8      public static void main(String[] args) {
9          // 1. Iniciar el contenedor de JPA mediante Persistence.createEntityManagerFactory y obtener el EntityManager.
10         EntityManagerFactory emf = Persistence.createEntityManagerFactory("FacturacionPU");
11         EntityManager em = emf.createEntityManager();
12
13         try {
14             // 2. Iniciar una transacción.
15             em.getTransaction().begin();
16
17             // 3. Instanciar los objetos necesarios (Usuario, PuntoVenta, ListaPrecioArticulo, etc.).
18             // Se crean y persisten primero porque son campos obligatorios (nullable = false) en AuditoriaApp y FacturaVenta.
19             Usuario usuario = new Usuario();
20             usuario.setUsuario("admin");
21             usuario.setClave("1234");
22             usuario.setNombre("Juan");
23             usuario.setApellido("Perez");
24             em.persist(usuario);
25
26             PuntoVenta puntoVenta = new PuntoVenta();
27             puntoVenta.setNumero(1);
28             puntoVenta.setFechaAlta(new Date());
29             puntoVenta.setFechaModificacion(new Date());
30             puntoVenta.setUsuarioCarga(usuario);
31             puntoVenta.setUsuarioModificacion(usuario);
32             em.persist(puntoVenta);
33
34             ListaPrecioArticulo listaPrecioArticulo = new ListaPrecioArticulo();
35             listaPrecioArticulo.setPrecioVenta(1500.0);
36             listaPrecioArticulo.setFechaAlta(new Date());
37             listaPrecioArticulo.setFechaModificacion(new Date());
38             listaPrecioArticulo.setUsuarioCarga(usuario);
39             listaPrecioArticulo.setUsuarioModificacion(usuario);
40             em.persist(listaPrecioArticulo);

```

```

41
42 // 4. Crear una cabecera de FacturaVenta[cite: 2].
43 FacturaVenta facturaVenta = new FacturaVenta();
44 facturaVenta.setNumero(1001L);
45 facturaVenta.setFechaEmision(new Date());
46 facturaVenta.setPuntoVenta(puntoVenta);
47 facturaVenta.setImporteTotal(3000.0);
48 facturaVenta.setEstado("EMITIDA");
49
50 // Atributos obligatorios por heredar de AuditoriaApp[cite: 2]
51 facturaVenta.setFechaAlta(new Date());
52 facturaVenta.setFechaModificacion(new Date());
53 facturaVenta.setUsuarioCarga(usuario);
54 facturaVenta.setUsuarioModificacion(usuario);
55
56 facturaVenta.setDetalles(new ArrayList<>());
57
58 // 5. Asignarle uno o más items FacturaVentaDetalle asociándolos bidireccionalmente[cite: 2].
59 FacturaVentaDetalle detalle1 = new FacturaVentaDetalle();
60 detalle1.setCantidad(2);
61 detalle1.setPrecioUnitario(1500.0);
62 detalle1.setImporteSubtotal(3000.0);
63 detalle1.setListaPrecioArticulo(listaPrecioArticulo);
64
65 // Configuración bidireccional: Asignar la factura al detalle y el detalle a la lista de la factura[cite: 2].
66 detalle1.setFactura(facturaVenta);
67 facturaVenta.getDetalles().add(detalle1);
68

```

```

68
69 // 6. Requisito clave: Persistir únicamente el objeto cabecera utilizando un solo llamado[cite: 2].
70 em.persist(facturaVenta);
71
72 // 7. Finalizar la transacción[cite: 2].
73 em.getTransaction().commit();
74 System.out.println(x: "Se ha insertado automáticamente la factura y sus detalles vinculados en la base de datos.");
75
76 } catch (Exception e) {
77     if (em.getTransaction().isActive()) {
78         em.getTransaction().rollback();
79     }
80     e.printStackTrace();
81 } finally {
82     // 8. Cerrar el EntityManager y EntityManagerFactory[cite: 2].
83     em.close();
84     emf.close();
85 }
86 }
87

```